Day 8: CI/CD Pipeline Integration

---

1. Introduction to CI/CD

Definition

CI/CD (Continuous Integration and Continuous Deployment) is a practice of automating code integration, testing, and deployment to ensure faster and reliable software delivery.

Key Concepts

Continuous Integration (CI)

- Automatically build and test code when changes are committed

Continuous Deployment (CD)

- Automatically deploy applications to environments

Benefits

- Faster releases
- Reduced manual errors
- Improved code quality
- Continuous feedback

Scenario

Manual deployment process

Problem:

- Slow releases
- Frequent errors

Solution:

- Implement CI/CD pipeline

Result:

- Automated and reliable deployments

Case Study

E-commerce Company

- Implemented CI/CD

Result:

- Reduced deployment time from hours to minutes
- Improved release frequency

---

2. CI/CD Tools for GKE

---

1. Cloud Build

Definition

Managed CI/CD service in GCP to build, test, and deploy applications.

---

2. GitHub Actions

Definition

CI/CD tool integrated with GitHub repositories.

---

3. GitLab CI

Definition

CI/CD tool integrated with GitLab for pipeline automation.

Example Workflow

1. Code push to repository
2. Build Docker image
3. Run tests
4. Push image to registry
5. Deploy to GKE

Scenario

Multiple developers committing code

Problem:

- Integration conflicts

Solution:

- Use CI pipelines

Result:

- Early detection of issues

Case Study

Tech Startup

- Uses GitHub Actions for CI
- Uses Cloud Build for deployment

Result:

- Automated end-to-end pipeline

---

3. GCP Secret Manager & Environment Variables

Definition

Secret Manager = Secure storage for sensitive data such as API keys, passwords, and tokens.

Environment Variables

- Used to pass configuration to applications
- Avoid hardcoding values

Example

export DB_PASSWORD=mysecret

Using Secret Manager

- Store secret securely

- Access during pipeline execution

Scenario

Credentials exposed in code

Problem:

- Security risk

Solution:

- Use Secret Manager

Result:

- Secure configuration management

Case Study

Financial Application

- Stores DB credentials in Secret Manager

Result:

- Secure and compliant system

---

4. Automating Build, Test, and Deploy

Pipeline Stages

1. Source
2. Build
3. Test
4. Push
5. Deploy

Example (Cloud Build YAML)

```
steps:
- name: 'gcr.io/cloud-builders/docker'
```

```
  args: ['build', '-t', 'asia-south1-docker.pkg.dev/$PROJECT_ID/my-repo/myapp:$SHORT_SHA', '.']

- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'asia-south1-docker.pkg.dev/$PROJECT_ID/my-repo/myapp:$SHORT_SHA']

- name: 'gcr.io/cloud-builders/kubectl'
  args:
  - 'apply'
  - '-f'
  - 'deployment.yaml'
```

Scenario

Frequent code changes

Problem:

- Manual build and deployment

Solution:

- Automate pipeline

Result:

- Faster and consistent releases

Case Study

SaaS Platform

- Fully automated pipeline

Result:

- Continuous delivery
- Minimal downtime

---

5. Notifications and Rollback Strategies

Notifications

- Email alerts
- Slack notifications
- Build status alerts

Rollback Strategies

1. Manual Rollback

kubectl rollout undo deployment myapp

2. Version-based Rollback

- Use image tags

3. Automated Rollback

- Trigger rollback on failure

Scenario

Failed deployment

Problem:

- New version breaks application

Solution:

- Rollback to previous version

Result:

- Service restored quickly

Case Study

Production Incident

- Deployment caused downtime

Solution:

- Immediate rollback

Result:

- Reduced impact on users

---

6. Hands-On: CI/CD Pipeline for GKE

Step 1: Connect Repository

- GitHub / GitLab repository

Step 2: Create Cloud Build Trigger

- Trigger on code push

Step 3: Build Image

```
docker build -t myapp .
```

Step 4: Push to Artifact Registry

```
docker push asia-south1-docker.pkg.dev/<project-id>/my-repo/myapp:v1
```

Step 5: Deploy to GKE

```
kubectl apply -f deployment.yaml
```

Step 6: Automate Entire Flow

- Configure pipeline YAML
- Enable trigger

Step 7: Verify Deployment

```
kubectl get pods
kubectl get services
```

End-to-End Case Study (Day 8 Summary)

Scenario: Automated Application Delivery

Problem:

- Manual deployments
- Inconsistent releases
- Security issues with credentials

Solution

1. Implement CI/CD pipeline
2. Use Cloud Build / GitHub Actions
3. Store secrets in Secret Manager
4. Automate build, test, and deploy
5. Configure notifications and rollback

Result

- Faster releases
- Secure pipelines
- Reliable deployments
- Reduced downtime